Informatik Projekt

Entwurf eines kamerabasierten Messprogramms in Python

Fach:	Angewandte Informatik
Dozent:	Prof. Bürkle
Studenten:	Tim Bosch
	Tizian Zimmer
	Jan Vagner
	Josef Rothermel
Kurs:	TMT24B2
Abgabedatum:	22.03.2026

Inhalt

Inhalt ... 2

Abbildungsverzeichnis ... 4

Tabellenverzeichnis ... 4

Verwendung von KI ... 5

Erklärung ... 5

1. Einleitung ... 6

2. Zielsetzung des Projekts ... 6

3. Aufgaben und Milestones ... 7

4. Theoretische Grundlagen ... 9

4.1 Kameramodell und Kamerakalibrierung ... 9

4.1.1 Mathematische Grundlagen ... 9

4.1.2 Durchführung der Kamerakalibrierung mit Schachbrettmuster ... 10

4.2 Marker-basierte Positions- und Lagebestimmung ... 11

4.2.1 Detektion der Marker ... 11

4.2.2 Auswahl und technische Spezifikation des Marker-Systems ... 12

4.2.3 Technische Codierung und Fehlersicherheit ... 13

4.2.4 Begründung der Systemwahl ... 14

4.2.5 Pose Schätzung der Marker ... 14

4.3 Messung von Abständen auf der Marker-Ebene ... 15

4.3.1 Pixelpunkt als Raumstrahl ... 15

4.3.2 Schnitt des Strahls mit der Marker-Ebene ... 16

4.3.3 Abstand zweier Punkte ... 17

5. Anforderungsanlyse und UseCase ... 18

5.2 UseCase ... 19

6. Softwarestruktur und Implementierung ... 20

6.1 Klassenstruktur ... 20

6.2 Hauptprogrammschleife ... 21

6.3 Kamerakomponente 22
6.4 Marker-Traking und Pose-Schätzung 23
6.5 Abstandsmessung 24
7.Test- und Validierungskonzept 24
7.1 Test und Validierungsaufbau 25
7.2 Funktionale Tests 26
7.2.1 Kamerastream-Modul 26
7.2.2 Marker-Tracking-Modul 26
7.2.3 Messwerkzeug 27
7.3 Geometrische Validierung 28
7.3.1 Referenzmessungen mit Meterstab 28
7.3.2 Vermessung eines Schachbrettmusters 29
7.4 Einflussfaktoren und Fehlerquellen 29
8.Ergebnisse und Auswertung 30
8.1 Funktionale Tests 30
8.1.1 Kamerastream-Modul 30
8.1.2 Marker-Tracking-Modul 30
8.1.3 Messwerkzeug 32
8.2 Geometrische Validierung 33
8.2.1 Referenzmessungen mit Meterstab 33
8.2.2 Vermessung eines Schachbrettmusters 34
9.Fazit und Ausblick 37
9.1 Fazit 37
9.2 Ausblick 38
10.Literaturverzeichnis 40
11.Bildquellen 40
12.Anhang 40

Abbildungsverzeichnis

Abbildung 1: GANTT Diagramm 7
Abbildung 2: Detektion der Ecken des Schachbrettmusters mithilfe des camera-calibration-using-opencv-python Repository (eigene Aufnahme, mit Logitech Brio100)..11
Abbildung 3 Detailansicht verschiedener Apriltag-Familien (apriltag, ohne Datum) 12
Abbildung 4 Freiheitsgrade 6 DOF (Strobl, 2025, S. 33 Abb. 33.1) 13
Abbildung 5: Veranschaulichung des Kameraursprungs (links), der Bildebene mit dem draufliegenden punkt p(x,y,1) und dem tatsächlichen Punkt M (X,Y,Z) (hier im Kamerakoordinatensystem) und des daraus resultierenden Vektors d (Szeliski, 2022, S. 58) 16
Abbildung 6: Kamerahalterung 25
Abbildung 7: Verschiede Markergrößen mithilfe eines Smartphones 30
Abbildung 8: Pose-Erkennung des Markers bei frontaler Betrachtung 31
Abbildung 9: Steilster Winkel eines Markers relativ zur Kamera welcher noch (mit einer gewissen Unsicherheit) als Kandidat erkannt wird 32
Abbildung 10: Durchführung einer einfachen Messung zum Testen der Messfunktionalität mithilfe eines Meterstabes 33
Abbildung 11: Horizontale Messung des Schachbrettmusters 35
Abbildung 12: Diagonale Messung des Schachbrettmusters 36

Tabellenverzeichnis

Tabelle 1: Anforderungsanlyse 18
Tabelle 2: Messwerttabelle Meterstab 34
Tabelle 3: Messwerttabelle Schachbrettmuster 35

Verwendung von KI

Im Rahmen der Erstellung dieser Arbeit wurde KI unterstützend eingesetzt. Die KI wurde dabei hauptsächlich zur Unterstützung bei der Formulierung, Strukturierung und sprachlichen Überarbeitung einzelner Textabschnitte verwendet. Außerdem wurde sie teilweise zum Auskommentieren von Programmcode genutzt und zur Erstellung des Bildes auf dem Deckblatt.

Die inhaltliche Planung, die Entwicklung des Messsystems, die Durchführung der Tests sowie die Auswertung der Ergebnisse erfolgten eigenständig durch die Autoren dieser Arbeit. Auch die Auswahl der verwendeten Methoden, die Implementierung der Software sowie die Interpretation der Messergebnisse wurden eigenständig erarbeitet.

Erklärung

Wir versichern hiermit, dass wir diese Ausarbeitung selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Vaihingen a.d. Enz 22.03.26

(Ort, Datum) (Unterschrift: Tim Bosch)

Karlsruhe, 22.03.26

(Ort, Datum) (Unterschrift: Josef Rothermel)

Karlsruhe 22.03.26 Jan Vagner

(Ort, Datum) (Unterschrift: Jan Vagner)

Karlsruhe 22.03.26 T. Zimmer

(Ort, Datum) (Unterschrift: Tizian Zimmer)

1. Einleitung

In der industriellen Produktion besteht häufig die Anforderung, Abstände und Geometrien von Bauteilen schnell und flexibel zu überprüfen, etwa im Rahmen der Qualitätssicherung. Klassische Messmittel liefern zwar präzise Ergebnisse, sind jedoch meist zeitaufwendig in der Handhabung oder nur eingeschränkt für schwer zugängliche Messstellen geeignet. Kamerabasierte Messsysteme bieten hier eine effiziente Alternative, da sie berührungslose Messungen ermöglichen und Messpunkte frei im Bild gewählt werden können. In industriellen Anwendungen kommen dabei häufig Mehrkamerasysteme, Laserlinien oder strukturierte Beleuchtung zum Einsatz, um eine hohe Messgenauigkeit und vollständige 3D-Erfassung zu gewährleisten.

Das im Rahmen des Projekts entwickelte Python-Programm dient der digitalen, berührungslosen Abstandsmessung und orientiert sich funktional an einem klassischen Messschieber. Aus Gründen der Einfachheit handelt es sich um einen Softwareprototyp, der mit einer einzelnen Kamera arbeitet. Durch Kamerakalibrierung und den Einsatz von Referenzmarkern (AprilTags) können dennoch reale Abstände auf einer definierten Ebene aus dem Livebild berechnet werden. Die Dokumentation beschreibt Aufbau und Umsetzung dieses Ansatzes.

2. Zielsetzung des Projekts

Ziel des Projekts ist die Entwicklung eines Python-basierten Softwareprototyps zur kameragestützten, berührungslosen Abstandsmessung auf Bauteilen. Das Programm soll die Bestimmung von Abständen direkt im Livebild ermöglichen, indem zwei Bildpunkte interaktiv ausgewählt und in reale metrische Größen überführt werden.

Hierzu sollen eine Kamerakalibrierung sowie Referenzmarker (AprilTags) eingesetzt werden, um die räumliche Lage einer Messebene relativ zur Kamera zu bestimmen. Auf dieser Grundlage sollen Bildkoordinaten in ein geeignetes Koordinatensystem transformiert und Abstände auf der Ebene berechnet werden. Das Programm soll auf einen stabilen Echtzeitbetrieb von etwa 30 Bildern pro Sekunde ausgelegt sein und reproduzierbare Messergebnisse bei bekannten Referenzabständen liefern.

3. Aufgaben und Milestones

Die Projektarbeit erfolgte phasenweise gemäß dem Zeitplan in Abbildung 1. Die Zusammenarbeit fand überwiegend online über Discord statt. Die Aufgabenverteilung wurde mündlich abgestimmt und flexibel angepasst. Neben der individuellen Bearbeitung einzelner Aufgaben wurde regelmäßig gemeinsam an Code, Dokumentation und Tests in Online-Treffen gearbeitet, um eine ständige Abstimmung im Team sicherzustellen.

Milestone	Bezeichnung	Fälligkeitsdatum	KW 3	KW 4	KW 5	KW 6	KW 7	KW 8	KW 9	KW 10	KW 11	KW 12	KW 13
1	Projektvorbereitung und Literaturrecherche	01.02.2026											
2	Softwareentwicklung und Integration	01.03.2026											
3	Dokumentation	15.03.2026											
4	Test, Validierung und Finalisierung	28.03.2026											
5	Projektabgabe	29.03.2026											

Abbildung 1: GANTT Diagramm

Phase 1: Projektvorbereitung und Literaturrecherche

- geeignete Fachliteratur (Kamerakalibrierung, Marker-basierte Pose-Schätzung, Projektionsgeometrie)
- Einarbeitung in relevante OpenCV-Funktionen
- Festlegung des Funktionsumfangs des Prototyps
- Grobstruktur der Dokumentation (Kapitel)

Meilenstein 1:

- Literaturbasis steht
- Dokumentationsstruktur festgelegt

Phase 2: Softwareentwicklung und Integration

- Implementierung des Kamerastreams
- Integration der Marker-Erkennung und Pose-Schätzung
- Umsetzung der interaktiven Abstandsmessung
- Erste interne Tests mit bekannten Referenzabständen um Funktion grob zu testen
- Kleine Anpassungen
- Auskommentieren des Codes

Meilenstein 2:

- Softwareprototyp funktionsfähig

- Messung im Livebild möglich

Phase 3: Technische Dokumentation

- Ausformulierung der theoretischen Grundlagen
    - Kalibrierung
    - Projektion
    - Ebenenschnitt und Abstandsberechnung
- Beschreibung der Softwarearchitektur und Module
- Fertiges Auskommentieren

Meilenstein 3:

- Rohfassung der Dokumentation abgeschlossen, Code vollständig auskommentiert

Phase 4: Test, Validierung und Finalisierung

- Durchführung gezielter Tests (Referenzlängen, Wiederholbarkeit)
- Bewertung der Messergebnisse
- Überarbeitung der Dokumentation

Meilenstein 4:

- Finale Version von Software und Dokumentation

Phase 5: Abgabe

- Endkontrolle

Abgabe des Projekts

4. Theoretische Grundlagen

4.1 Kameramodell und Kamerakalibrierung

Eine digitale Kamera bildet dreidimensionale Punkte des Raumes auf zweidimensionale Pixelkoordinaten ab. Diese Abbildung ist perspektivisch, wodurch Tiefeninformationen verloren gehen und reale Abstände aus einem einzelnen Bild nicht eindeutig bestimmt werden können. Als Hilfestellung hierfür dienen die intrinsische Kameramatrix und die Verzerrungskoeffizenten (Zhang, 2000, S. 1330).

4.1.1 Mathematische Grundlagen

Ein 3D-Punkt im Raum wird in homogenen Koordinaten beschrieben als:

$$M = (X, Y, Z, 1)^{\mathrm{T}}$$

$$(die\ 1\ dient\ der\ vereinfachten\ Multiplikation)$$

Der zugehörige Bildpunkt lautet:

$$m = (u, v, 1)^{\mathrm{T}}$$

$$(die\ 1\ dient\ der\ vereinfachten\ Multiplikation)$$

Nach dem Projektionsmodell nach Zhang gilt:

$$s \cdot m = A \cdot [R|t] \cdot M$$

Dabei bezeichnet:

- A die intrinsische Kameramatrix
- R und t die Rotation und Translation zwischen Welt- und Kamerakoordinatensystem $[R\,|\,t] = [\,r_1, r_2, r_3, t]$

(Rotationsmatrix 3x3, Translationsvektor 3x1)

- s ist ein Skalierungsfaktor

Die intrinsische Matrix besitzt die Form:

$$A = \begin{bmatrix} \alpha, \gamma, u^0 \\ 0, \beta, v^0 \\ 0, 0, 1 \end{bmatrix}$$

mit:

- α und β effektive Brennweiten in Pixelrichtung
- γ Scherungsparameter
- (u_0, v_0) Hauptpunkt

4.1.2 Durchführung der Kamerakalibrierung mit Schachbrettmuster

Zur Bestimmung der intrinsischen Matrix und den Verzerrungskoeffizienten wird ein planares Referenzmusters in Form eines Schachbretts verwendet. Das Schachbrett besitzt eine dem Kalibrierungs-Algorithmus bekannte geometrische Struktur mit regelmäßig angeordneten Eckpunkten. Für jede Aufnahme werden die Bildkoordinaten dieser Schachbrettecken automatisch detektiert.

Da alle Punkte des Schachbretts in einer Ebene liegen, können bei mehreren unterschiedlich ausgerichteten Aufnahmen die bekannten 3D-Musterpunkte M eindeutig den jeweils detektierten Bildpunkten m zugeordnet werden. Für jede Aufnahme sind somit die räumlichen Koordinaten der Schachbrettecken im Musterkoordinatensystem sowie deren zugehörige Pixelkoordinaten im Bild bekannt. Daraus können die intrinsische Kameramatrix und die Verzerrungskoeffizienten berechnet werden (Zhang, 2000, S. 1331–1332)

Da diese Methode der Bestimmung mathematisch komplex und aufwändig ist haben wir uns dafür entschieden, für die die praktische Durchführung dieser Kalibrierung das OpenCV-basierte GitHub-Repository *camera-calibration-using-opencv-python* zu verwenden, um den Fokus auf das eigentliche Messprogramm legen zu können. Bei diesem werden mehrere Aufnahmen des Schachbretts in unterschiedlichen Orientierungen mithilfe des Python-Skriptes "collect_images" eingelesen, die Eckpunkte extrahiert und anschließend das Python-Skript "calibrate_camera" zur Parameterberechnung aufgerufen.

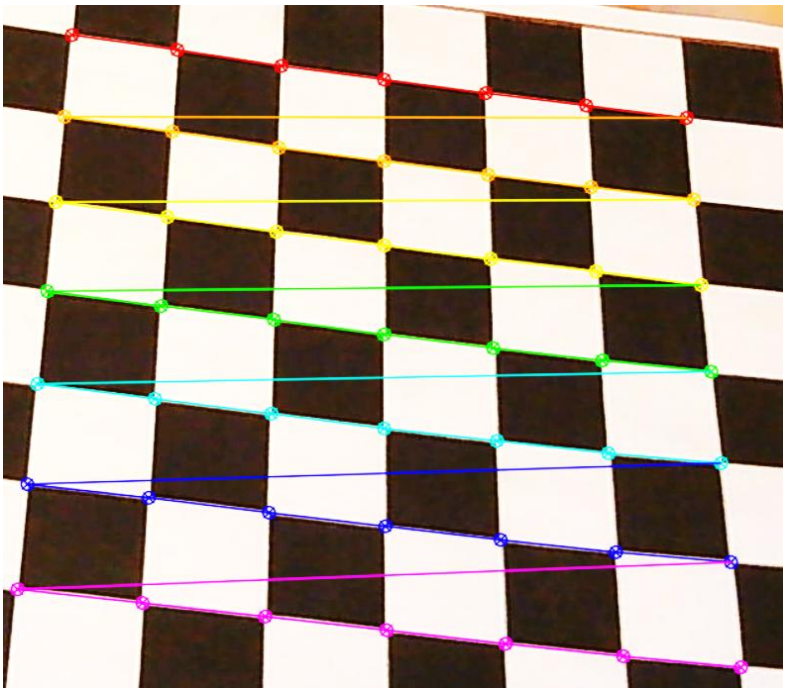


Abbildung 2: Detektion der Ecken des Schachbrettmusters mithilfe des camera-calibration-using-opencv-python Repository (eigene Aufnahme, mit Logitech Brio100)

Die ermittelten Parameter werden zunächst im Output-Verzeichnis des Kalibrierungsskripts gespeichert und anschließend in das Projektverzeichnis des Messprogramms verschoben. Dort liegen sie in Form von NumPy-Dateien (camera_matrix.npy, dist_coeffs.npy) und werden beim Start des Messprogramms geladen und verwendet.

Erst durch diese Kalibrierung wird die Abstandsmessung mithilfe einer 2D-Kamera überhaupt erst möglich.

4.2 Marker-basierte Positions- und Lagebestimmung

4.2.1 Detektion der Marker

Zur Bestimmung einer Referenzebene im Raum werden AprilTags eingesetzt. Dabei handelt es sich um quadratische, binär codierte Marker mit eindeutig identifizierbarer ID. Die Detektion der AprilTags erfolgt mithilfe der OpenCV-Aruco-Bibliothek. Zunächst wird das Kamerabild mit cv2.cvtColor in ein Graustufenbild umgewandelt, da die Marker-Erkennung auf Helligkeitsinformationen basiert.

Anschließend wird ein cv2.aruco.ArucoDetector verwendet. Der Detektor sucht nach quadratischen Kandidatenstrukturen mit hohem Kontrast. Für jede gefundene Struktur werden die vier Eckpunkte bestimmt und das innere Binärmuster ausgelesen. Dieses wird

mit dem gewählten AprilTag-Dictionary (im Projekt: 25h9) abgeglichen. Nur bei eindeutiger Übereinstimmung wird der Marker akzeptiert.

Als Ergebnis liegen die Pixelkoordinaten der vier Markerecken sowie die zugehörige ID vor und bilden die Grundlage für die anschließende Pose-Schätzung.

4.2.2 Auswahl und technische Spezifikation des Marker-Systems

Nachdem die grundlegende Detektion der Markerstrukturen im Bild erfolgt ist, ist es für die Präzision des Messsystems entscheidend, welches spezifische Marker-System verwendet wird. Im Rahmen dieses Projekts wurde die Entscheidung für AprilTags der Familie 25h9 getroffen. Diese Wahl basiert auf einem Abwägen zwischen Detektionsreichweite, Recheneffizienz und der Robustheit gegenüber Fehlinterpretationen.

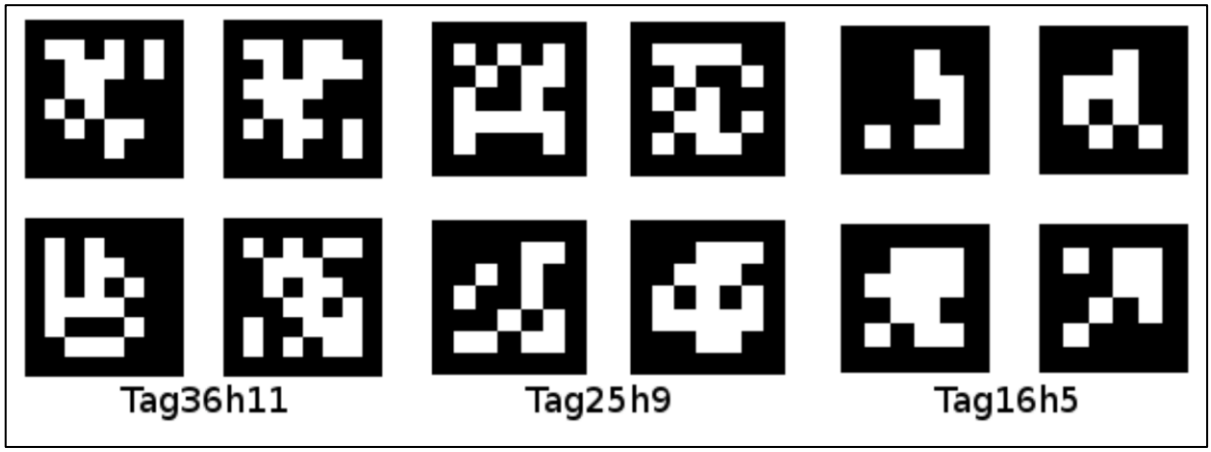


Abbildung 3 Detailansicht verschiedener Apriltag-Familien (apriltag, *ohne Datum)*

Die Entscheidung für künstliche Merkmale (Fiducials) folgt dem wissenschaftlichen Ansatz, eine kontrollierte Messumgebung zu schaffen und die Komplexität der Objekterkennung zu reduzieren. Im Gegensatz zu komplexeren Systemen, wie aktiv leuchtenden LED-Markern, bieten die hier verwendeten AprilTags den Vorteil, dass sie kostengünstig gedruckt werden können und dennoch eine vollständige Lokalisierung mit sechs Freiheitsgraden (6 DOF - Degrees of Freedom) (vgl. Abbildung 4) aus einem einzigen Bild ermöglichen. Der Marker übernimmt damit die Funktion einer stabilen

geometrischen Basis, die eine präzise Überführung von Bildkoordinaten in den dreidimensionalen Raum erlaubt.

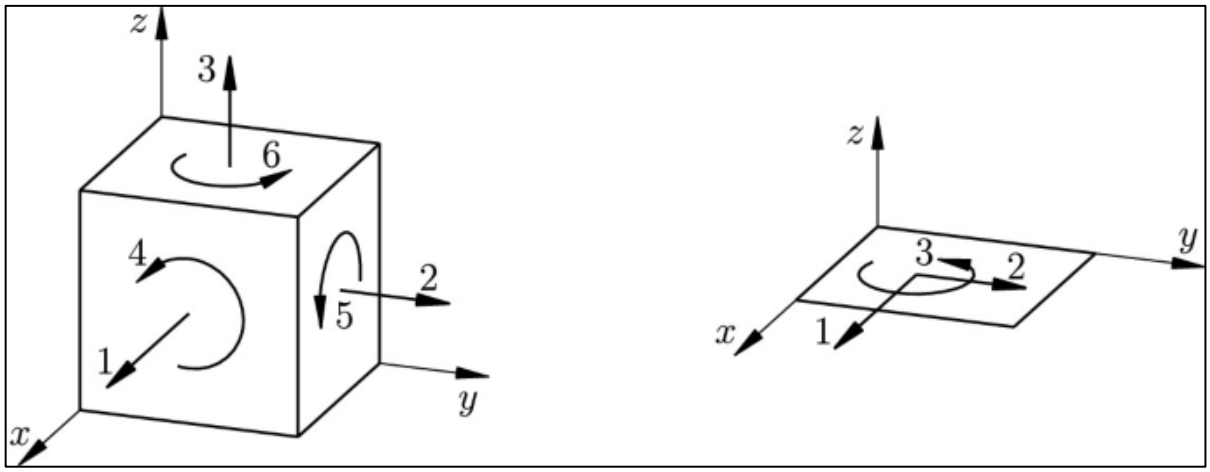


Abbildung 4 Freiheitsgrade 6 DOF (Strobl, 2025, S. 33 Abb. 33.1)

4.2.3 Technische Codierung und Fehlersicherheit

Ein wesentliches Merkmal von AprilTags ist ihre binäre Codierung. Während einfache Marker-Systeme lediglich ein Muster zur Identifikation nutzen, basieren AprilTags auf mathematisch optimierten Dictionaries. Die Bezeichnung 25h9 bedeutet, dass das innere Datenfeld aus 25 Bits (einem 5x5-Raster) besteht und eine Hamming-Distanz von 9 aufweist.

Nach der Definition von Olson (2011) werden diese Marker als klassische Lexicodes beschrieben, die durch die Bit-Anzahl n und die minimale Hamming-Distanz d zwischen zwei beliebigen Codewörtern parametrisiert sind. Die mathematische Sicherheit gegen Fehlinterpretationen lässt sich dabei direkt aus der Distanz d ableiten: Ein System kann laut Olson [(d−1)/2] Bitfehler korrigieren und d/2 Fehler sicher erkennen.

Für das im Projekt verwendete 25h9-Dictionary bedeutet dies konkret, dass bis zu 4 einzelne Bitfehler [(9−1)/2] innerhalb des 5x5-Rasters korrigiert werden können. Ein weiterer entscheidender Sicherheitsaspekt ist die Robustheit gegenüber Rotationen: Die Codegenerierung nach Olson stellt sicher, dass die minimale Hamming-Distanz von 9 auch dann gewahrt bleibt, wenn der Marker um 90, 180 oder 270 Grad gedreht detektiert wird.(vgl. Olson, 2011, S. 3403)

4.2.4 Begründung der Systemwahl

Im Vergleich zu komplexeren Familien wie 36h11 bietet die 25h9-Familie bei identischer physischer Größe des Markers größere einzelne Daten-Bits (vgl. Abbildung 3). Dies führt dazu, dass der Marker im Kamerabild auch bei geringerer Auflösung oder größeren Entfernungen noch sicher in seine binären Bestandteile zerlegt werden kann. Da das Ziel des Projekts ein stabiler Echtzeitbetrieb bei 30 FPS ist, unterstützt die Wahl dieses Dictionaries die Performance, da die Validierung des Bit-Musters weniger Rechenressourcen beansprucht als bei höher auflösenden Markern.

Zudem wurde in den funktionalen Tests nachgewiesen, dass dieses System eine Detektion bis zu einem sehr steilen Betrachtungswinkel ermöglicht. Die Robustheit der 25h9-Codierung stellt sicher, dass die für die anschließende Pose-Schätzung notwendigen Eckpunkte erst dann extrahiert werden, wenn die ID-Validierung zweifelsfrei erfolgt ist.

4.2.5 Pose Schätzung der Marker

Nach der erfolgreichen Detektion liegen die Pixelkoordinaten der vier Markerecken im Bild vor. Da die reale Kantenlänge des Markers bekannt ist, können die entsprechenden 3D-Koordinaten der Eckpunkte im Markerkoordinatensystem definiert werden. Für einen planaren Marker gilt:

$$M = (X, Y, 0,1)^{\mathrm{T}}$$

und

$$m = (u, v, 1)^{\mathrm{T}}$$

Die Beziehung zwischen Markerpunkt und Bildpunkt wird durch das Projektionsmodell nach Zhang beschrieben:

$$s \cdot m = A \cdot [R|t] \cdot M$$

Da die intrinsische Matrix A aus der Kalibrierung bekannt ist, können die Parameter R und t des Markers bestimmt werden. Die OpenCV-Aruco-Bibliothek stellt hierfür mit der Funktion cv2.aruco.estimatePoseSingleMarkers eine einfach nutzbare Lösung bereit.

Dieser Funktion werden die detektierten Eckpunkte des Markers, die reale Markergröße, die zuvor bestimmte intrinsische Matrix A und die Verzerrungskoeffizienten übergeben. Als Ergebnis liefert OpenCV Arurco die Rotation und Translation des Markers relativ zur Kamera.

Die Rotation wird dabei zunächst als Rodrigues-Rotationsvektor (rvec) ausgegeben. Für die weitere geometrische Verarbeitung kann dieser mithilfe der Funktion cv2.Rodrigues in eine 3×3-Rotationsmatrix umgewandelt werden.

Das bildet die Grundlage für die Beschreibung der Marker-Ebene im Kamerakoordinatensystem. Aus der Rotationsmatrix R kann die Orientierung der Marker-Ebene bestimmt werden. Der Normalenvektor der Marker-Ebene ergibt sich aus der dritten Spalte der Rotationsmatrix bzw. durch

$$n = R \cdot (0,0,1)^{\mathrm{T}}$$

Der Translationsvektor t beschreibt die Position des Marker-Ursprungs im Kamerakoordinatensystem:

$$t = (t^1, t^2, t^3)^{\mathrm{T}}$$

Damit ist die Lage der Ebene, welche der Marker aufspannt, eindeutig definiert. Diese Information wird im weiteren Verlauf genutzt, um Bildpunkte in reale 3D-Punkte auf der Marker-Ebene zu überführen und daraus metrische Abstände zu berechnen.

4.3 Messung von Abständen auf der Marker-Ebene

4.3.1 Pixelpunkt als Raumstrahl

Wird im Kamerabild ein Punkt mit der Maus angeklickt, liegt zunächst lediglich dessen Pixelkoordinate im 2D Raum vor:

$$m = (u, v, 1)^{\mathrm{T}}$$

Dieser kann mithilfe der intrinsischen Matrix und den Verzerrungskoeffizienten ins Kamerakoordinatensystem überführt werden.:

$$p = A^{-1} \cdot m$$

Aus dem Verbindungsvektor des Kameraursprungs und dem errechneten Punkt ergibt sich ein Richtungsvektor im Kamerakoordinatensystem:

$$d = (x, y, 1)^{\mathrm{T}}$$

Dieser Vektor beschreibt die Richtung eines Strahls im Kamerakoordinatensystem. Der Punkt der den Pixel "ausgelöst" hat, könnte also beliebig weit von Ursprung entfernt auf der Geraden mit diesem Richtungsvektor liegen (Szeliski, 2022, S. 54–61).

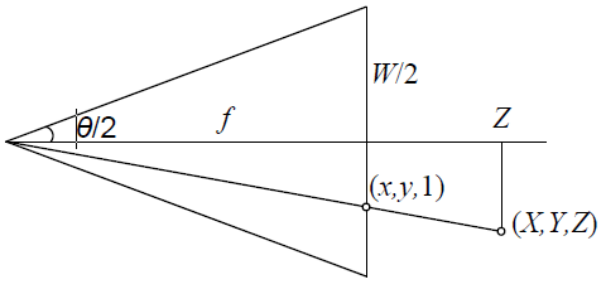


Abbildung 5: Veranschaulichung des Kameraursprungs (links), der Bildebene mit dem draufliegenden punkt p(x,y,1) und dem tatsächlichen Punkt M (X,Y,Z) (hier im Kamerakoordinatensystem) und des daraus resultierenden Vektors d (Szeliski, 2022, S. 58)

4.3.2 Schnitt des Strahls mit der Marker-Ebene

Die Marker-Ebene ist durch R und t vollständig definiert. Ein Punkt auf der Ebene ist der Markerursprung:

$$P_0 = t$$

Der Raumstrahl-Vektor kann parametrisch beschrieben werden durch:

$$X(\lambda) = \lambda \cdot d$$

So werden alle auf dem Strahl liegende Punkte beschrieben.

Der Schnittpunkt zwischen Strahl und Ebene ergibt sich durch einsetzen in die Ebenengleichung:

$$n^{\mathrm{T}} \cdot (X - P_0) = 0$$

$$n^T * (\lambda \cdot d - P_0) = 0$$

$$\lambda \cdot n^T d - n^T P_0 = 0$$

$$\lambda \cdot n^T d = n^T P_0$$

$$\lambda = \frac{n^T P_0}{n^T d}$$

Der reale 3D-Punkt auf der Marker-Ebene ergibt sich somit zu:

$$X = \lambda \cdot d$$

Also:

$$X = \left(\frac{n^T P_0}{n^T d}\right) \cdot d$$

4.3.3 Abstand zweier Punkte

Wenn nun der Abstand zwischen zwei Punkten berechnet werden soll, ergibt sich dieser aus dem Betrag des Verbindungsvektors:

$$Abstand = |X_2 - X_1|$$

Da alle Berechnungen im metrischen Maßstab der Markergröße erfolgen, ergibt sich der Abstand direkt in realen Längeneinheiten. Durch diese Mathematischen Grundlagen können mithilfe eines 2D Bildes, eines AprilTags, der Kameramatrix und den Verzerrungskoeffizienten Maße im dreidimensionalen Raum ermittelt werden.

5. Anforderungsanlyse und UseCase

In diesem Kapitel werden die Anforderungen an das System strukturiert dargestellt. Sie bilden die Grundlage für Entwicklung und Bewertung des Prototyps.Es wird zwischen funktionalen Anforderungen (Was soll das System können?) und nicht-funktionalen Anforderungen (Wie gut soll es das tun?) unterschieden. Die folgende Tabelle enthält alle identifizierten Anforderungen mit Nummer, Art, Beschreibung und Priorität.

Tabelle 1: Anforderungsanlyse

Nummer	Art	Anforderung	Wert, Beschreibung	Priorität
1	Funktional	Kamerastream anzeigen	Live-Videostream einer externen Kamera mit 1920x1080 @ 30 FPS	Muss
2	Funktional	Kamerakalibrierung durchführen	Kalibrierung mittels Schachbrett o.ä und Speicherung der Parameter (Umsetzung nach möglichkeit mit externer Bibliothek	Muss
3	Funktional	Kalibrierungsdaten laden	Laden gespeicherter Kamera-Matrix A und Verzerrungskoeffizienten (incl. Path Handling)	Muss
4	Funktional	Marker-Erkennung	Erkennung von AprilTag-Markern der Familie 25h9 im Livebild	Muss
5	Funktional	Markerecken bestimmen	Bestimmung der vier Pixelkoordinaten der Markerecken	Muss
6	Funktional	Pose-Schätzung	Berechnung von Rotation R und Translation t des Markers relativ zur Kamera	Muss
7	Funktional	Markerachsen visualisieren	Einblendung der ArUrco 3D-Achsenprojektion im Kamerabild	Soll
8	Funktional	Punktselektion per Maus	Benutzer kann zwei Pixelpunkte im Livebild anklicken	Muss
9	Funktional	Projektion auf Markerebene	Rückprojektion der Pixelpunkte auf die Ebene Z=0 des Markers	Muss
10	Funktional	Abstandsberechnung	Berechnung des realen Abstands in mm zwischen zwei projizierten Punkten	Muss
11	Funktional	Messergebnis anzeigen	Darstellung der Distanz als Textoverlay im Livebild	Muss
12	Funktional	Gitter	Ebene des Markers durch Gitterstruktur sichtbar machen	Soll
13	Funktional	Messung zurücksetzen	Zurücksetzen der ausgewählten Punkte per Tasteneingabe	Soll
14	Funktional	Benutzersteuerung	Bedienung über Maus und definierte Tastaturbefehle	Muss
15	Nicht-funktional	Messgenauigkeit	Abweichung ≤ ±10 % bei Referenzabständen im Messbereich	Muss
16	Nicht-funktional	Echtzeitfähigkeit	Systemlatenz < 250 ms bei laufender Marker-Erkennung (Bedienung soll sich flüssig anfühlen)	Muss
17	Nicht-funktional	Robustheit	System darf bei temporärem Marker-Verlust nicht abstürzen oder undefinierte Zustände generieren	Muss
19	Nicht-funktional	Estetik	Übersichtliche Darstellung, klare Visualisierung von Punkten und Messlinie	Soll
20	Nicht-funktional	Wartbarkeit	Modularer Codeaufbau (Kamera, Marker, Messlogik durch Klassen getrennt)	Muss
21	Nicht-funktional	Erweiterbarkeit	Erweiterung auf mehrere Marker oder 3D-Messungen möglich	Soll
22	Nicht-funktional	Plattform	Lauffähig unter Windows 10/11	Muss
24	Nicht-funktional	Performance	Lauffähig auf Standard-Laptop mit USB-Webcam (keine GPU erforderlich, Leistungsoptimierung)	Soll

5.2 UseCase

Der zentrale Use Case des entwickelten Systems besteht in der Messung realer Abstände auf einer definierten Markerebene. Der Benutzer startet das Programm, richtet die Kamera auf den Referenzmarker und wählt anschließend zwei Punkte im Livebild per Mausklick aus. Das System erkennt den Marker im Kamerabild, berechnet dessen räumliche Pose relativ zur Kamera und projiziert die ausgewählten Pixelpunkte auf die definierte Marker-Ebene. Auf Grundlage dieser Rückprojektion wird der reale Abstand zwischen den beiden Punkten berechnet und direkt im Livebild dargestellt.

Durch dieses Vorgehen können Objekte oder Bauteile schnell und berührungslos vermessen werden. Das System ist dabei als prototypisches Werkzeug zu verstehen und nicht als hochpräzises Messinstrument. Aufgrund der begrenzten Genauigkeit eines einzelnen Kamerasystems können Messabweichungen im Bereich weniger Prozent auftreten. Das System eignet sich daher insbesondere für Anwendungen, bei denen ungefähre Abmessungen ausreichen oder eine schnelle Orientierung über Größenverhältnisse benötigt wird.

Ein möglicher Einsatzbereich liegt im Automobilsektor, insbesondere bei Reparatur- und Restaurationsarbeiten. In Werkstätten müssen häufig Maße von Bauteilen bestimmt werden, um Ersatzteile anzufertigen, oder um Bauteile miteinander zu vergleichen. Gerade bei älteren Fahrzeugen besteht häufig das Problem, dass bestimmte Ersatzteile vom Hersteller nicht mehr produziert werden oder nicht mehr verfügbar sind. In solchen Fällen müssen einzelne Komponenten, oder Dichtungen selbst hergestellt werden.

Ein Beispiel wäre die Anfertigung von Öl- oder Wasserdichtungen. Hierbei wird häufig eine vorhandene Dichtung als Vorlage verwendet. In der Regel die zerstörte Dichtung. Mit dem kamerabasierten Messsystem kann eine solche Dichtung auf die Markerebene gelegt und anschließend im Bild vermessen werden. Die relevanten Abstände können direkt am Bildschirm bestimmt werden, wodurch sich der Prozess der Maßaufnahme vereinfachen lässt. Die ermittelten Werte dienen dabei als Orientierung für die weitere handwerkliche Bearbeitung.

Auch im Karosseriebau kann ein solches System unterstützend eingesetzt werden. Wenn beschädigte Blechteile ersetzt werden müssen, wird häufig zunächst die benötigte Form ermittelt, um anschließend ein passendes Ersatzblech zuzuschneiden. In der Praxis wird dazu häufig mit Klebeband gearbeitet, auf dem die Form des benötigten Blechs markiert wird. Das System könnte hier genutzt werden, um die ungefähren Abmessungen einer Fläche schnell zu bestimmen, oder verschiedene Punkte eines Bauteils zueinander in

Beziehung zu setzen. Dadurch lässt sich beispielsweise abschätzen, welche Größe ein benötigtes Blechstück haben muss.

Neben handwerklichen Anwendungen kann das System auch als allgemeines Hilfsmittel zur schnellen Vermessung von Objekten eingesetzt werden. Beispielsweise lassen sich Abstände zwischen Bohrungen, Kanten oder Bauteilmerkmalen grob bestimmen, ohne dass ein Messschieber direkt am Objekt angesetzt werden muss. Dies kann insbesondere bei schwer zugänglichen Stellen hilfreich sein.

Darüber hinaus kann das System auch zur groben geometrischen Analyse von Bauteilen genutzt werden. Wenn mehrere Punkte auf einer Oberfläche bestimmt werden, lassen sich daraus weitere geometrische Beziehungen ableiten. Beispielsweise können durch Messung mehrerer Abstände Rückschlüsse auf Formen oder Winkel zwischen Bauteilen gezogen werden. Die Ergebnisse dienen hierbei vor allem der Orientierung oder Vorbereitung weiterer Arbeitsschritte.

Grundsätzlich lässt sich das entwickelte System als optisches Messhilfsmittel verstehen. Kamerasysteme können genutzt werden, um geometrische Informationen aus Bilddaten zu gewinnen und daraus reale Größen abzuschätzen. In der vorliegenden Form dient das System vor allem dazu, schnell und flexibel Abstände zu bestimmen, ohne dass spezielle Messgeräte benötigt werden. Die Einsatzmöglichkeiten liegen daher vor allem in Situationen, in denen eine schnelle, einfache und berührungslose Messung ausreichend ist.

6. Softwarestruktur und Implementierung

6.1 Klassenstruktur

Das Messsystem ist modular aufgebaut und gliedert sich in drei zentrale Funktionsbereiche: Bildaufnahme, Marker-Tracking mit Pose-Schätzung sowie die eigentliche Abstandsmessung auf der Marker-Ebene. Diese Aufgaben sind jeweils in eigenständigen Klassen gekapselt. Die Hauptschleife des Programms verbindet diese Module zu einer kontinuierlich arbeitenden Verarbeitungskette.

Ziel dieser Struktur ist eine klare Trennung der Funktion: Die Kamera liefert ausschließlich Bilddaten, der Tracker bestimmt die geometrische Lage des Markers im Raum, und das Messmodul verarbeitet Benutzereingaben und berechnet daraus reale Abstände.

6.2 Hauptprogrammschleife

Das Programm läuft in der klassischen Echtzeit-Loop-Struktur:

1. **FPS aktualisieren**
   Über CamStream.fps_calc() wird etwa jede Sekunde eine FPS-Schätzung berechnet und als Text eingeblendet.
2. **Frame erfassen mit**
   CamStream.run() liest einen neuen Frame aus cv2.VideoCapture.
3. **Marker detektieren**
   AprilTagTracker.detect() konvertiert den Frame in ein Graustufenbild und ruft anschließend ArucoDetector.detectMarkers() auf.
4. **Pose schätzen (nur wenn Marker vorhanden)**
   Sobald IDs vorhanden sind, wird aus den detektierten Eckpunkten mit cv2.aruco.estimatePoseSingleMarkers() die Pose bestimmt: Rotationsvektor rvec und Translationsvektor tvec. (Praktisch Umsetzung des Projektionsmodell aus Kap. 4.2.2)
5. **Pose für Messung setzen**
   Das Messwerkzeug bekommt die Pose des ersten erkannten Markers via PlaneMeasurementTool.set_pose(rvecs[0], tvecs[0]).
   Damit ist die Marker-Ebene im Kamerakoordinatensystem definiert (Kap. 4.2.2 / 4.3.2).
6. Overlays zeichnen
   cv2.drawFrameAxes() projiziert die 3D-Achsen des Markers ins Bild.
7. Messung aktualisieren
   PlaneMeasurementTool.update_and_draw() zeichnet Klickpunkte, verbindet sie und zeigt den berechneten Abstand an (in cm).
   Mathematisch entspricht das dem Ablauf aus Kap. 4.3: Pixelpunkt → Raumstrahl → Schnitt mit Ebene → Abstand.
8. Anzeige + Eingabe
   Anzeige über cv2.imshow(), Eingabe über cv2.waitKey(1), c: Messpunkte löschen (measurer.clear()), ESC oder q: Programm beenden
   Zusätzlich wird geprüft, ob das Fenster geschlossen wurde (cv2.getWindowProperty).

6.3 Kamerakomponente

Die Kamerakomponente ist für die Initialisierung der Kamera sowie für die kontinuierliche Bereitstellung der Bilddaten verantwortlich. Im Programm wird diese Funktion durch die Klasse CamStream umgesetzt. Sie bildet den Einstiegspunkt der Bildverarbeitung und stellt in jedem Schleifendurchlauf ein aktuelles Kamerabild für die weiteren Verarbeitungsschritte bereit.

Die Kamera wird unter Windows mithilfe des Backends Microsoft Media Foundation (cv2.CAP_MSMF) geöffnet. In praktischen Tests mit der verwendeten USB-Webcam hat sich dieses Backend als besonders stabil erwiesen. Andere verfügbare Backends führten teilweise zu instabilen Bildraten. Aus diesem Grund wurde Microsoft Media Foundation als Backend für das Projekt mit der logitech Brio 100 gewählt. Während der Initialisierung der Kamera werden mehrere Parameter festgelegt. Dazu gehören insbesondere die gewünschte Bildauflösung, die Bildrate sowie der verwendete Videocodec. In dem Projekt wird eine Auflösung von 1920 × 1080 Pixeln bei einer Zielbildrate von 30 Bildern pro Sekunde verwendet. Als Codec wird Motion JPEG (MJPG) eingestellt. Dieser Codec hat sich in den Tests als vorteilhaft erwiesen, da er bei hohen Auflösungen häufig eine stabilere Bildrate ermöglicht und gleichzeitig die CPU-Belastung geringhält. Teilweise wurden aber auch durch andere Codecs ähnliche Ergebnisse erzielt. Nach dem Öffnen der Kamera werden zunächst mehrere Frames gelesen, um der Kamera Zeit für automatische Anpassungen wie Belichtung oder Autofokus zu geben. Dieser sogenannte „Warmup“-Schritt verhindert, dass die ersten Frames mit falschen Belichtungswerten verarbeitet werden. Während des laufenden Programms liefert die Kamerakomponente in jedem Schleifendurchlauf ein aktuelles Frame als NumPy-Array. Zusätzlich wird eine einfache Messung der aktuellen Bildrate durchgeführt. Hierfür wird gezählt, wie viele Frames innerhalb einer Sekunde verarbeitet werden. Der berechnete FPS-Wert wird anschließend direkt im Kamerabild eingeblendet, sodass während des Betriebs jederzeit sichtbar ist, mit welcher Geschwindigkeit das System arbeitet. Die Kamerakomponente übernimmt ausschließlich die Aufnahme und Anzeige der Bilddaten. Eine geometrische Verarbeitung der Bildinformationen findet in dieser Klasse bewusst nicht statt. Diese Trennung sorgt für eine klare Struktur des Programms, da Bildaufnahme, Marker-Erkennung und Abstandsmessung in voneinander unabhängigen Komponenten umgesetzt sind.

6.4 Marker-Traking und Pose-Schätzung

Die Marker-Erkennung sowie die Bestimmung der Marker-Pose werden im Programm durch die Klasse AprilTagTracker umgesetzt. Diese Klasse kapselt die komplette Verarbeitung rund um die Marker-Erkennung und stellt Methoden für Detektion, Pose-Schätzung und Visualisierung bereit.

Die Marker-Erkennung basiert auf der OpenCV-Aruco-Implementierung für AprilTags der Familie 25h9. Beim Initialisieren der Klasse wird zunächst das passende Marker-Dictionary geladen. Zusätzlich werden Detektorparameter erzeugt und ein ArucoDetector erstellt, der später zur eigentlichen Marker-Erkennung verwendet wird. Außerdem werden die zuvor bestimmten Kameraparameter, also die intrinsische Kameramatrix sowie die Verzerrungskoeffizienten, an die Klasse übergeben und gespeichert. Die eigentliche Marker-Erkennung erfolgt in der Methode detect(). Dabei wird das aktuelle Kamerabild zunächst in ein Graustufenbild umgewandelt. Anschließend wird der ArucoDetector aufgerufen, der im Bild nach gültigen Markern sucht. Für jeden erkannten Marker werden die vier Eckpunkte im Bild bestimmt und als Pixelkoordinaten zurückgegeben. Optional können die erkannten Marker direkt im Bild visualisiert werden. Diese Pixelkoordinaten entsprechen den Bildpunkten

$$m = (u, v, 1)^T$$

aus den theoretischen Grundlagen. Da die reale Markergröße bekannt ist, können die zugehörigen 3D-Punkte im Marker-Koordinatensystem definiert werden. Mit cv2.aruco.estimatePoseSingleMarkers werden aus diesen 2D–3D-Korrespondenzen unter Verwendung der intrinsischen Kameramatrix A und der Verzerrungskoeffizienten die extrinsischen Parameter R und t berechnet. Damit wird praktisch die in Kapitel 4 beschriebene Projektionsbeziehung $s \cdot m = A \cdot [R \mid t] \cdot M$ angewendet.

Aus der bestimmten Pose ergibt sich die Lage der Marker-Ebene im Kamerakoordinatensystem. Diese Ebene wird im weiteren Verlauf des Programms als geometrische Referenz verwendet. Die berechneten Rotations- und Translationsvektoren werden im Hauptprogramm an das Messwerkzeug übergeben, das auf dieser Grundlage die Rückprojektion der Bildpunkte auf die Marker-Ebene durchführen kann.

Die Klasse stellt damit die notwendigen geometrischen Informationen bereit, die für die spätere Abstandsmessung benötigt werden.

6.5 Abstandsmessung

Die Abstandsmessung ist im Programm in der Klasse PlaneMeasurementTool umgesetzt. Diese Klasse verarbeitet die Benutzerinteraktion und übernimmt die geometrische Berechnung der Abstände zwischen zwei Punkten auf der Marker-Ebene.

Die Auswahl der Messpunkte erfolgt direkt im Kamerabild. Hierfür wird ein OpenCV-Maus-Callback verwendet, der über die Funktion cv2.setMouseCallback() an das Anzeigefenster gebunden wird. Sobald der Benutzer mit der linken Maustaste in das Bild klickt, wird die entsprechende Pixelposition gespeichert. Die Klickpunkte werden intern als Liste von Pixelkoordinaten (x, y) gespeichert. Sobald zwei Punkte vorhanden sind, beginnt die eigentliche Abstandsbewertung. Ein Rechtsklick oder das Setzen eines neuen Messpunkts setzt die vorherige Messung zurück. Ein angeklickter Bildpunkt liegt zunächst nur als Pixelkoordinate

$$m = (u, v, 1)^T$$

vor. Mithilfe von cv2.undistortPoints wird dieser unter Berücksichtigung der intrinsischen Matrix und der Verzerrungskoeffizienten in Kamerakoordinaten überführt. Entsprechend Kapitel 4.3 entsteht daraus ein Richtungsvektor, der einen Raumstrahl vom Kameraursprung beschreibt.

Mit der zuvor bestimmten Marker-Pose (Rotation R und Translation t) wird anschließend der Schnittpunkt dieses Strahls mit der Marker-Ebene berechnet. Dabei wird die in den theoretischen Grundlagen hergeleitete Schnittbedingung zwischen Strahl und Ebene angewendet. Anschließend wird mithilfe des Betrags des Verbindungsvektors der Abstand bestimmt. Da durch estimatePoseSingleMarkers das Kamerakoordinatensystem bereits in Metern aufgespannt ist, ist keine zusätzliche Skalierung nötig. Die Messlinie sowie der berechnete Abstand werden anschließend direkt im Livebild visualisiert.

7.Test- und Validierungskonzept

Zur Sicherstellung der Funktionsfähigkeit und Messgenauigkeit des Systems wurde ein mehrstufiges Test- und Validierungskonzept angewendet. Ziel war es, sowohl die technische Stabilität der Software als auch die geometrische Korrektheit der Abstandsmessung zu überprüfen.

7.1 Test und Validierungsaufbau

Um beim Testen und Validieren des kamerabasierten Messsystems möglichst konstante, sichere Bedingungen zu gewährleisten, ist ein reproduzierbarer Versuchsaufbau essenziell. Ein zentrales Element hierfür ist die feste und wiederholbar positionierbare Montage der Kamera.

Für diesen Zweck wurde eine eigene Messeinrichtung entwickelt, in der die Kamera mithilfe eines Stativs präzise ausgerichtet werden kann und trotzdem mobil ist. Da handelsübliche Halterungen nicht den spezifischen Anforderungen des Projekts entsprechen, wurde eine maßgeschneiderte Kamerahalterung konstruiert und mittels 3D-Druck gefertigt.

Die Halterung ist so gestaltet, dass sie direkt in das Stativ eingesetzt werden kann und gleichzeitig die Kamera sicher trägt. Bei der Konstruktion wurde besonderer Wert auf eine einfache Geometrie gelegt, um eine hohe Druckqualität und Stabilität zu gewährleisten. Gedruckt wurde das Bauteil mit einem Flashforge 3 Lite unter Verwendung von schwarzem PLA-Filament.

Abbildung 6: Kamerahalterung

Sowohl die Kamera als auch die gedruckte Halterung können schnell vom Stativ gelöst werden. Dies reduziert das Transportvolumen und erleichtert den mobilen Einsatz des Messsystems. Zudem ermöglicht die modulare Bauweise den Austausch einzelner Komponenten im Falle eines technischen Defekts – beispielsweise kann die Kamera separat ausgetauscht werden, ohne dass das komplette Stativ oder der Adapter erneut gefertigt werden muss.

Eine technische Zeichnungsableitung des 3D-gedruckten Bauteils befindet sich im Anhang. Diese enthält alle relevanten Maße und Details der Fertigung

7.2 Funktionale Tests

Zunächst wurden funktionale Tests durchgeführt, um die einzelnen Softwaremodule isoliert zu prüfen.

7.2.1 Kamerastream-Modul

Als erster Schritt wurde das Kamerastream-Modul separat getestet. Dafür wurde eine eigene Testdatei verwendet, in der zunächst nur die Kamera initialisiert und das Livebild angezeigt wurde. Marker-Erkennung und Messfunktionen waren zu diesem Zeitpunkt noch nicht integriert. Ziel dieses Tests war es, die stabile Initialisierung der Kamera sowie den kontinuierlichen Kamerastream zu überprüfen.

Die Kamera wurde unter Windows mit dem Microsoft Media Foundation Backend (MSMF) gestartet. Dieses Backend wurde gewählt, da es sich in den Vorversuchen als besonders performant und stabil erwiesen hat und daher auch im finalen Programm verwendet wird. Zur Überprüfung der Performance wurde die in Kapitel 6 beschriebene FPS-Messung eingesetzt. Dabei wurde die Bildrate während des laufenden Kamerastreams kontinuierlich erfasst. Zusätzlich wurde überprüft, ob die gewünschte Auflösung korrekt gesetzt wird und der Stream ohne Unterbrechungen läuft. Während der Tests zeigte sich außerdem, dass die Kamera beim Start möglichst bereits auf die beleuchtete Szene ausgerichtet sein sollte. Wird das Backend gestartet, während die Kamera zunächst auf eine dunkle Umgebung zeigt, kann die automatische Belichtungsregelung ungeeignete Werte einstellen, was später zu überbelichteten Bildern führen kann. Darüber hinaus wurde beobachtet, dass bei schnellen Bewegungen im Bild Bewegungsunschärfe auftreten kann. Für die geplante Anwendung ist dies jedoch unkritisch, da die Messungen in der Regel bei ruhiger Kameraposition und statischen Objekten erfolgen. Abschließend wurde geprüft, ob das Programm sauber beendet werden kann. Dabei wurde kontrolliert, ob die Kamera korrekt freigegeben wird und alle OpenCV-Fenster ordnungsgemäß geschlossen werden, um Probleme beim erneuten Start zu vermeiden.

7.2.2 Marker-Tracking-Modul

Das Marker-Tracking-Modul wurde getestet, um die zuverlässige Detektion der AprilTags sowie die Stabilität der Pose-Schätzung zu überprüfen. Dazu wurde das Kamerabild

kontinuierlich ausgewertet und untersucht, ob der Marker unter verschiedenen Bedingungen korrekt erkannt wird.

Ein Schwerpunkt der Tests lag auf der Detektion des Markers bei unterschiedlichen Entfernungen und Betrachtungswinkeln zur Kamera. Hierbei wurde überprüft, ob der Marker auch bei variierendem Abstand oder geneigter Kamera erkannt wird. Zur Durchführung der Tests wurde der Marker zusätzlich auf einem Smartphone dargestellt. Dadurch konnte die Größe des Markers flexibel verändert werden, indem der Marker auf dem Display vergrößert oder verkleinert wurde. Auf diese Weise ließ sich untersuchen, wie sich unterschiedliche Markergrößen auf die Erkennung auswirken. Eine beispielhafte Darstellung der Markeranzeige ist in Abbildung X zu sehen. Darüber hinaus wurden unterschiedliche Beleuchtungssituationen getestet, um zu beobachten, wie sich Änderungen der Helligkeit auf die Marker-Erkennung auswirken. Zusätzlich wurde die Stabilität der ID-Erkennung untersucht. Dabei wurde überprüft, ob der Marker über mehrere Frames hinweg konsistent mit derselben ID erkannt wird. Zur visuellen Kontrolle der Pose-Schätzung wurde das Koordinatensystem des Markers mithilfe der Funktion cv2.drawFrameAxes() im Kamerabild dargestellt. Diese Darstellung dient als qualitative Plausibilitätskontrolle, da sie zeigt, ob die berechnete Pose des Markers geometrisch plausibel im Bild liegt.

7.2.3 Messwerkzeug

Im nächsten Schritt wurde das Messmodul getestet, welches für die eigentliche Abstandsmessung auf der Marker-Ebene verantwortlich ist. Ziel dieser Tests war es zu überprüfen, ob die Benutzerinteraktion korrekt funktioniert und ob die Berechnung der Abstände grundsätzlich wie vorgesehen durchgeführt wird.

Dazu wurden im Kamerabild verschiedene Punkte auf der Marker-Ebene ausgewählt. Die Auswahl erfolgte über einen Maus-Callback, der beim Anklicken eines Bildpunktes dessen Pixelkoordinate speichert. Sobald zwei Punkte ausgewählt wurden, berechnet das System den Abstand zwischen den beiden Punkten und stellt diesen im Livebild dar. Während der Tests wurde überprüft, ob die angeklickten Punkte korrekt gespeichert werden und ob die Messlinie zwischen den beiden Punkten im Bild dargestellt wird. Zusätzlich wurde getestet, ob eine neue Messung gestartet werden kann, indem die zuvor gesetzten Punkte zurückgesetzt werden. Darüber hinaus wurde kontrolliert, ob das System nur dann eine Messung durchführt, wenn ein Marker erkannt wurde und somit eine gültige Referenzebene vorhanden ist. Dadurch wird verhindert, dass Messungen ohne definierte Geometrie durchgeführt werden. Die Tests wurden mit verschiedenen Punktpositionen

auf der Marker-Ebene durchgeführt, um sicherzustellen, dass die Berechnung unabhängig von der Position im Bild funktioniert.

7.3 Geometrische Validierung

Neben den funktionalen Tests wurde auch die geometrische Genauigkeit des Systems überprüft. Ziel dieser Tests war es zu untersuchen, wie stark die mit dem System berechneten Abstände von realen Referenzmaßen abweichen.

Hierzu wurden mehrere Testreihen durchgeführt, bei denen bekannte Abstände gemessen und mit den vom System berechneten Werten verglichen wurden. Die Tests konzentrierten sich auf typische Anwendungssituationen des Systems. Als Referenzobjekte wurden ein Meterstab sowie ein Schachbrettmuster verwendet. Beide Objekte besitzen bekannte geometrische Strukturen und eignen sich daher gut zur Überprüfung der Messgenauigkeit.

Während der Tests wurde außerdem darauf geachtet, die Messungen an unterschiedlichen Positionen im Kamerabild durchzuführen, um mögliche Einflüsse der Bildposition auf die Messergebnisse zu untersuchen.

7.3.1 Referenzmessungen mit Meterstab

Zur Überprüfung der grundlegenden Messgenauigkeit wurden Referenzmessungen mit einem Meterstab durchgeführt. Der Meterstab wurde dabei flach auf die Marker-Ebene gelegt, sodass sich Maßstab und Marker in derselben Ebene befanden.

Anschließend wurden verschiedene Strecken entlang des Meterstabs im Kamerabild ausgewählt. Die Messpunkte wurden dabei so gesetzt, dass sie möglichst genau auf den entsprechenden Markierungen des Maßstabs lagen. Die realen Referenzwerte wurden direkt am Meterstab abgelesen.

Die Tests konzentrierten sich zunächst auf kleinere Abstände im Bereich weniger Zentimeter, da diese für den vorgesehenen Anwendungsfall besonders relevant sind. Zusätzlich wurden auch größere Abstände ausgewählt, um zu untersuchen, wie sich die Messabweichung bei zunehmender Distanz verhält. Um die Konsistenz der Messergebnisse zu überprüfen, wurde der Meterstab während der Tests mehrfach innerhalb des Kamerabildes verschoben. Dabei wurden identische Referenzabstände an unterschiedlichen Positionen im Bild erneut gemessen. Auf diese Weise konnte überprüft werden, ob sich die Messergebnisse bei veränderter Position im Bild stabil verhalten. Für jede Messung wurde der vom System berechnete Abstand mit dem tatsächlichen

Referenzmaß verglichen und die Abweichung dokumentiert. Die Ergebnisse dieser Messungen werden im Kapitel Ergebnisse und Auswertung dargestellt.

7.3.2 Vermessung eines Schachbrettmusters

Zur weiteren Überprüfung der geometrischen Genauigkeit wurde zusätzlich ein Schachbrettmuster vermessen. Ein Schachbrett besitzt eine regelmäßige Struktur aus gleich großen Quadraten und eignet sich daher gut als Referenzobjekt für wiederholbare Abstandsmessungen.

Das Schachbrett wurde flach auf die Marker-Ebene gelegt, sodass sich beide Objekte in derselben geometrischen Ebene befanden. Anschließend wurden verschiedene Abstände zwischen den Eckpunkten der Schachbrettfelder im Kamerabild ausgewählt. Dabei wurden sowohl horizontale als auch vertikale Strecken zwischen benachbarten Feldern gemessen. Zusätzlich wurden auch diagonale Strecken über mehrere Felder hinweg bestimmt. Dadurch lässt sich überprüfen, ob die geometrische Rückprojektion unabhängig von der Messrichtung konsistente Ergebnisse liefert. Darüber hinaus wurden auch längere Strecken über mehrere Felder hinweg gemessen, um zu untersuchen, wie sich mögliche Messabweichungen mit zunehmender Distanz verändern. Die realen Referenzwerte ergeben sich aus der bekannten Kantenlänge der einzelnen Schachbrettfelder. Die gemessenen Werte wurden anschließend mit diesen Referenzmaßen verglichen und dokumentiert. Die Ergebnisse dieser Messreihe werden im Kapitel Ergebnisse und Auswertung dargestellt.

7.4 Einflussfaktoren und Fehlerquellen

Während der Tests wurden mehrere Faktoren identifiziert, die die Messgenauigkeit beeinflussen können.

Ein möglicher Einflussfaktor ist die Kamerakalibrierung. Werden bei der Kalibrierung nicht genügend unterschiedliche Aufnahmen verwendet oder werden die Schachbrettecken ungenau erkannt, können kleine Fehler in den berechneten Kameraparametern entstehen. Diese wirken sich anschließend direkt auf die Rückprojektion der Bildpunkte und damit auf die berechneten Abstände aus.

Auch im Randbereich des Kamerabildes können leichte Verzerrungseffekte auftreten. Trotz der Korrektur durch die berechneten Verzerrungskoeffizienten können Messungen nahe am Bildrand daher etwas ungenauer sein als Messungen im zentralen Bildbereich.

Der größte Einflussfaktor ist jedoch die manuelle Auswahl der Messpunkte im Bild. Da die Punkte per Maus gesetzt werden, ist es schwierig, exakt die gewünschte Position zu treffen. Bereits kleine Abweichungen von wenigen Pixeln können zu leicht veränderten Messergebnissen führen.

8. Ergebnisse und Auswertung

8.1 Funktionale Tests

8.1.1 Kamerastream-Modul

Im Testbetrieb zeigte sich, dass der Kamerastream stabil arbeitet und die eingestellte Auflösung von 1920×1080 zuverlässig erreicht wird. Mithilfe der implementierten FPS-Messung konnte im Livebetrieb eine Bildrate von etwa 30 Bildern pro Sekunde erreicht werden. Diese Bildrate ist ausreichend, um eine flüssige Darstellung des Kamerabildes sowie eine reaktionsschnelle Benutzerinteraktion im Messprogramm zu gewährleisten.

Die Verwendung des Microsoft Media Foundation Backends (MSMF) erwies sich dabei als besonders stabil und performant. Während der Tests traten keine relevanten Bildaussetzer oder Verbindungsprobleme auf, sodass das Modul als geeignet für den vorgesehenen Echtzeitbetrieb bewertet werden kann.

8.1.2 Marker-Tracking-Modul

Die Tests zeigten, dass der Marker bei unterschiedlichen Entfernungen und Beleuchtungsbedingungen zuverlässig erkannt wird. Auch bei variierenden Markergrößen, die über das Smartphone-Display dargestellt wurden, funktionierte die Detektion stabil (siehe Abbildung .

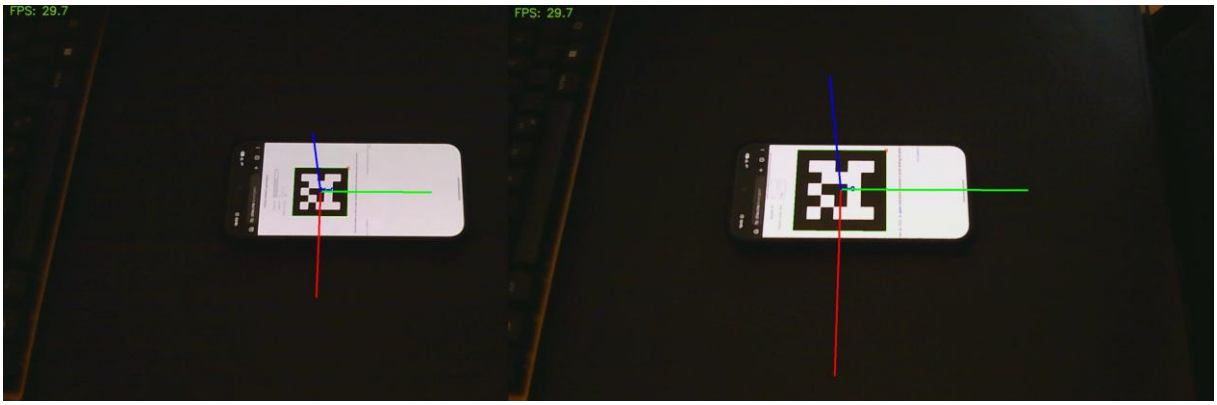


Abbildung 7: Verschiede Markergrößen mithilfe eines Smartphones

Die ID-Erkennung blieb während der Tests konsistent über mehrere Frames hinweg. Für das entwickelte System ist die konkrete ID jedoch nicht kritisch, da im Programm lediglich der erste erkannte Marker als Referenz für die Messung verwendet wird.

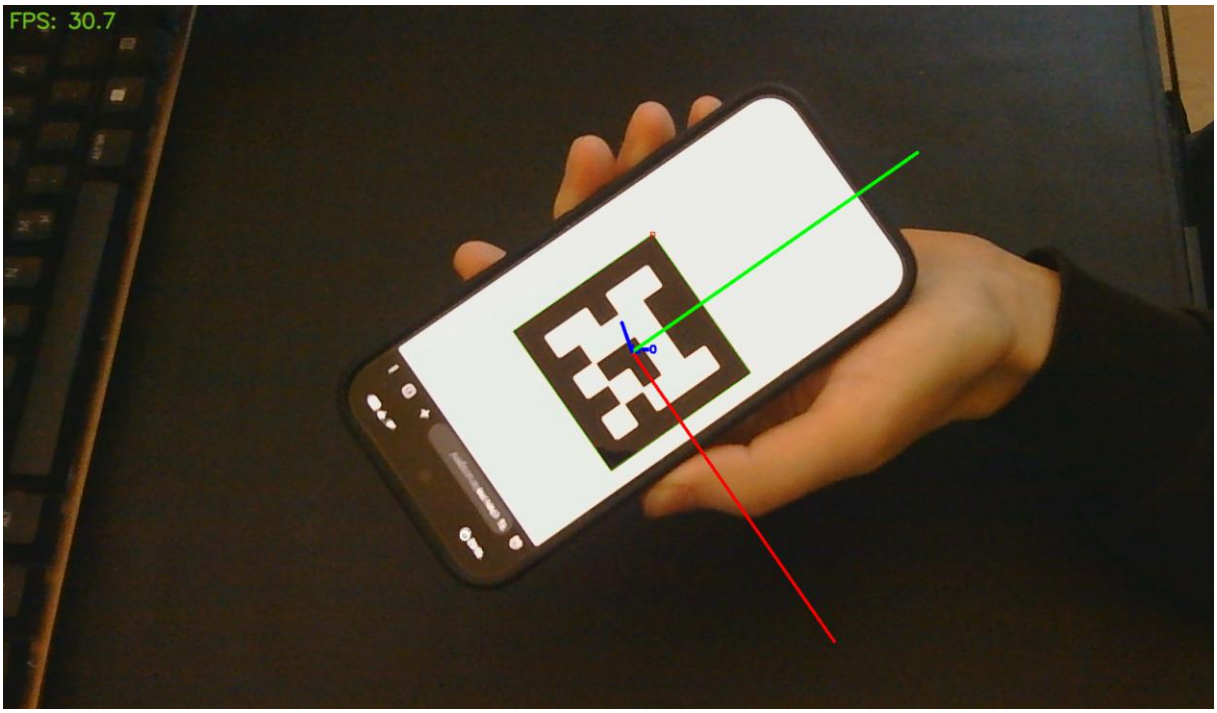


Abbildung 8: Pose-Erkennung des Markers bei frontaler Betrachtung

Bei sehr steilen Betrachtungswinkeln wurde beobachtet, dass die Erkennung teilweise schwieriger wird und die Pose-Schätzung weniger stabil ist. Für den vorgesehenen Anwendungsfall stellt dies jedoch kein wesentliches Problem dar, da der Marker in der Praxis üblicherweise relativ frontal zur Kamera ausgerichtet ist.

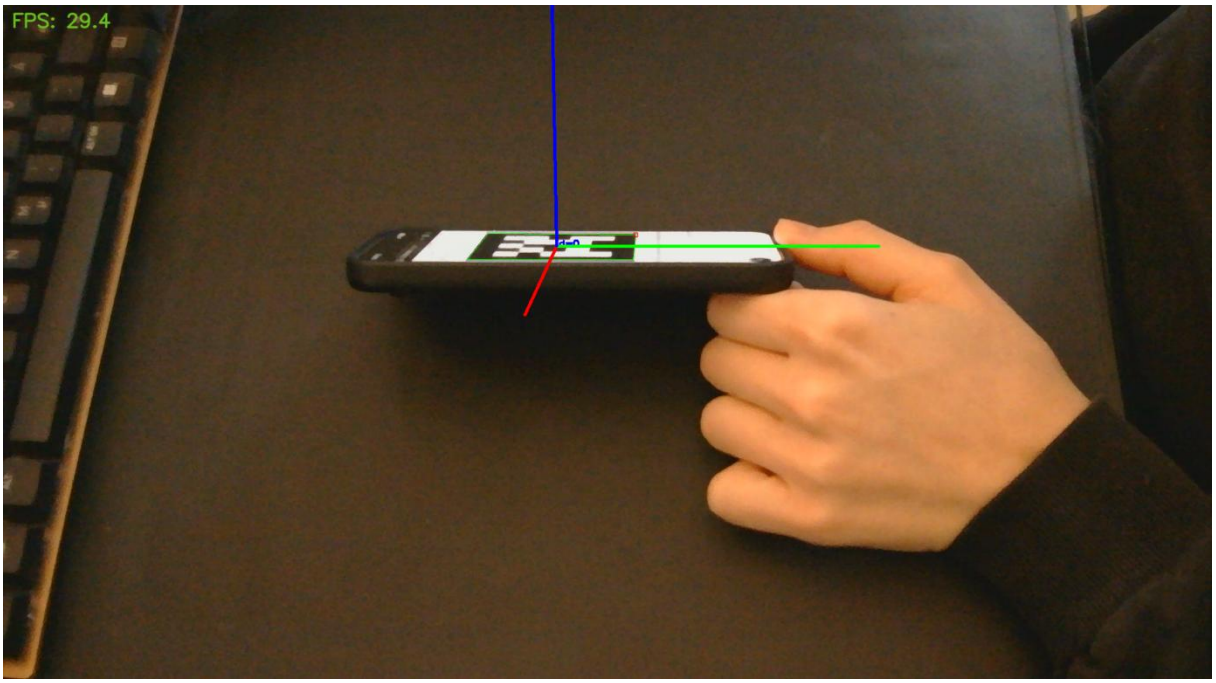


Abbildung 9: Steilster Winkel eines Markers relativ zur Kamera welcher noch (mit einer gewissen Unsicherheit) als Kandidat erkannt wird

Das projizierte Achsenkreuz lag während der Tests stabil auf dem Marker und bewegte sich plausibel mit dessen Lage im Raum. Dabei konnte jedoch ein leichtes Zittern des Achsenkreuzes beobachtet werden. Dies bestätigt qualitativ, dass die berechnete Marker-Pose korrekt bestimmt wird.

8.1.3 Messwerkzeug

Die Tests zeigten, dass das Messwerkzeug zuverlässig arbeitet und die Benutzerinteraktion stabil funktioniert. Die ausgewählten Punkte wurden korrekt registriert und nach Auswahl von zwei Punkten wurde der berechnete Abstand unmittelbar im Livebild dargestellt.

Auch das Zurücksetzen der Messpunkte funktionierte wie vorgesehen, sodass neue Messungen problemlos gestartet werden konnten. Zudem wurde überprüft, dass eine

Messung nur dann durchgeführt wird, wenn zuvor ein Marker erkannt wurde und somit eine gültige Referenzebene vorhanden ist.

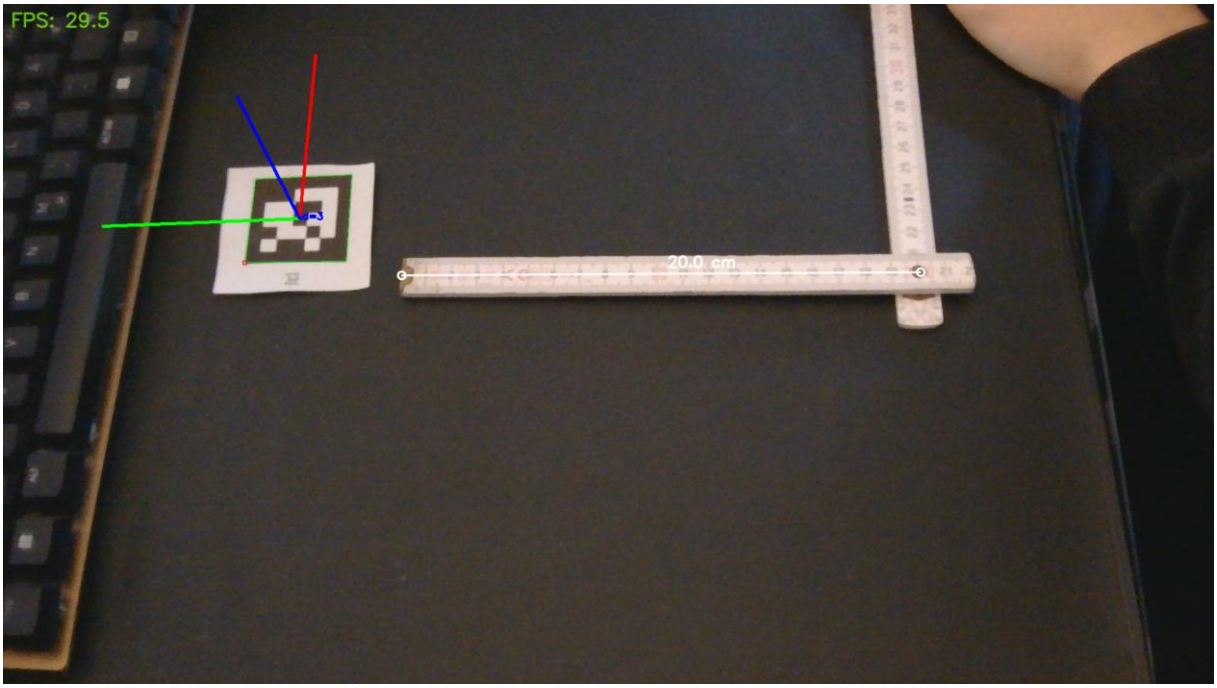


Abbildung 10: Durchführung einer einfachen Messung zum Testen der Messfunktionalität mithilfe eines Meterstabes

Wird der Marker aus dem Sichtfeld der Kamera entfernt, verschwindet auch die dargestellte Messung im Bild. Dadurch wird verhindert, dass Messwerte angezeigt werden, obwohl keine gültige Referenzebene mehr vorhanden ist.

Wird der Marker anschließend wieder im Kamerabild erkannt, erscheinen auch die zuvor gesetzten Messpunkte und die Messanzeige wieder. Voraussetzung hierfür ist jedoch, dass derselbe Marker mit unveränderter Größe verwendet wird. Da die reale Markergröße als Referenzmaß für die Berechnung der Abstände dient, würde eine Änderung der Markergröße zu falschen Messergebnissen führen.

Während der Tests zeigte sich außerdem, dass die Darstellung der Messlinie sowie die Anzeige des berechneten Abstandes stabil im Bild eingeblendet werden und sich korrekt aktualisieren, wenn neue Punkte ausgewählt werden.

8.2 Geometrische Validierung

8.2.1 Referenzmessungen mit Meterstab

Die durchgeführten Referenzmessungen zeigen insgesamt eine gute Übereinstimmung zwischen den realen Abständen des Meterstabs und den vom System berechneten Messwerten. Bei kleinen Abständen zwischen 1 cm und 5 cm stimmen die gemessenen

Werte in vielen Fällen exakt mit dem Referenzmaß überein oder weichen nur geringfügig davon ab.

Tabelle 2: Messwerttabelle Meterstab

Echtes Maß (cm)	1	2	3	4	5	6	7	8	9	10
Links Oben	1	2	3,1	4	5	6,1	7	8,1	9	10,1
Mittig	1	2,1	3	4	5,1	6	7,1	8	9,1	10
Rechts Unten	1,1	2	3	4,1	5	6	7,1	8,1	9	10,1

Mit zunehmender Distanz treten vereinzelt kleine Abweichungen auf. Diese liegen überwiegend im Bereich von etwa 1 mm. Besonders bei Abständen über 7 cm wurden vereinzelt solche Abweichungen beobachtet (siehe Tabelle 2). Auch beim Vergleich der verschiedenen Bildpositionen (links oben, mittig und rechts unten) zeigen die Messwerte ein weitgehend konsistentes Verhalten. Ein signifikanter Einfluss der Position im Bild konnte nicht festgestellt werden. Die beobachteten Abweichungen lassen sich hauptsächlich durch zwei Faktoren erklären. Zum einen führen kleine Schwankungen in der Pose-Schätzung des Markers zu minimalen Veränderungen der berechneten Marker-Ebene. Zum anderen entsteht durch die manuelle Auswahl der Messpunkte mit der Maus eine gewisse Ungenauigkeit, da die Punkte nicht immer exakt auf den Markierungen des Meterstabs gesetzt werden können. Der größte Einflussfaktor auf die Messabweichung ist die manuelle Auswahl der Messpunkte im Kamerabild. Da die Punkte per Maus gesetzt werden, ist es schwierig, die Markierungen des Meterstabs exakt zu treffen. Bereits kleine Abweichungen von wenigen Pixeln können zu einer leichten Veränderung des berechneten Abstandes führen. Daher ist davon auszugehen, dass die beobachteten Abweichungen hauptsächlich durch die Benutzerinteraktion und nicht durch die mathematische Berechnung selbst entstehen. Insgesamt liegen die Abweichungen im Bereich weniger Prozent und sind für den vorgesehenen Einsatz als orientierendes kamerabasiertes Messwerkzeug ausreichend.

8.2.2 Vermessung eines Schachbrettmusters

Die Vermessung des Schachbrettmusters bestätigt die Ergebnisse der vorherigen Referenzmessungen mit dem Meterstab. Die gemessenen Abstände stimmen in den meisten Fällen gut mit den erwarteten Referenzmaßen überein (siehe Tabelle 3).

Tabelle 3: Messwerttabelle Schachbrettmuster

Echtes Maß (cm)	2,5	5	7,5	10	12,5	15	17,5	20
Horizontal								
1 Reihe	2,5	5	7,5	10	12,3	14,8	17,3	19,9
4 Reihe	2,5	5	7,4	10	12,4	14,9	17,4	19,8
8 Reihe	2,5	5,1	7,5	10	12,5	15	17,4	19,9
Vertikal								
1 Spalte	2,5	5	7,5	10	12,4	14,9	17,4	19,7
4 Spalte	2,5	5	7,6	10	12,5	15	17,5	19,7
8 Spalte	2,5	5,1	7,5	10	12,5	15	17,5	20
Echtes Maß (cm)	2,5	5	7,5	10	12,5	15	17,5	20
Diagonal	28	14						
1 Reihe, 1 Spalte bis 8 Reihe 8 Spalte	28,1	14						

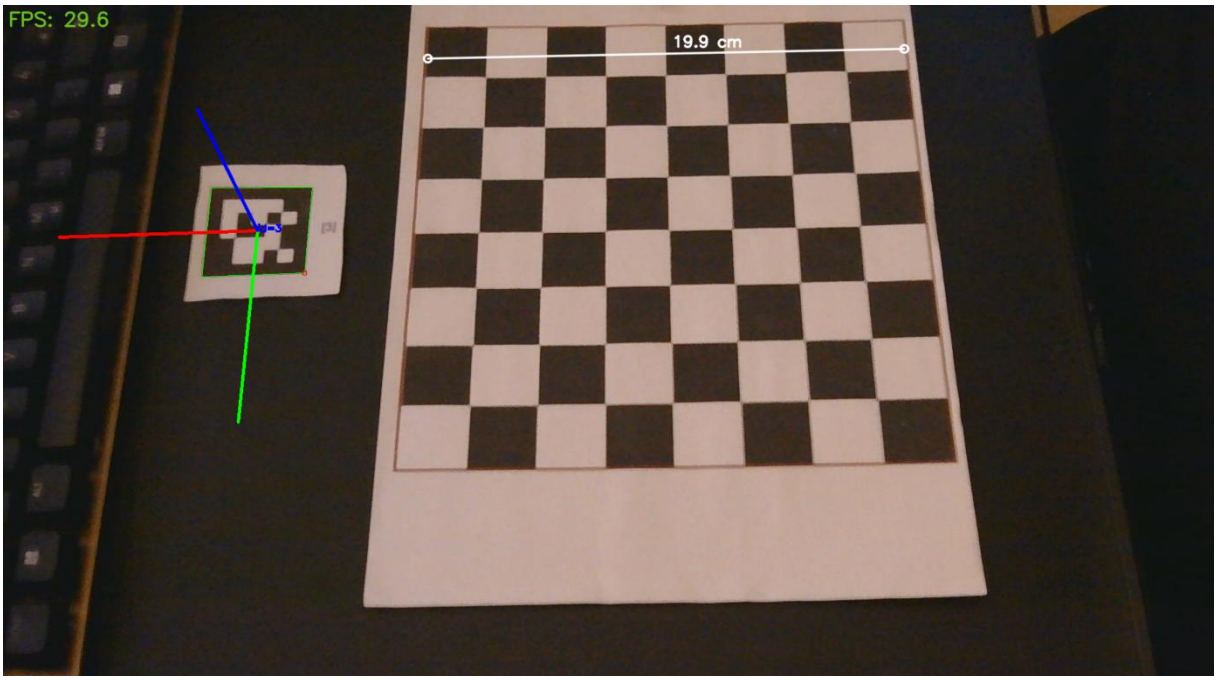


Abbildung 11: Horizontale Messung des Schachbrettmusters

Bei kleineren Messstrecken bis etwa 10 cm liegen die gemessenen Werte sehr nahe am Referenzmaß. Mit zunehmender Distanz treten vereinzelt kleine Abweichungen auf, die überwiegend im Bereich von etwa 1 mm liegen.

Sowohl bei horizontalen als auch bei vertikalen Messungen zeigen die Ergebnisse ein konsistentes Verhalten. Ein signifikanter Unterschied zwischen den verschiedenen Positionen im Bild konnte nicht festgestellt werden. Dies deutet darauf hin, dass die geometrische Rückprojektion der Bildpunkte auf die Marker-Ebene über das gesamte Bild hinweg stabil funktioniert.

Um dies weiter zu validieren, wurden zusätzlich diagonale Messungen durchgeführt. Diese sind besonders geeignet, um mögliche geometrische Verzerrungen oder

Ungenauigkeiten in der Rückprojektion zu erkennen, da sie nicht entlang der horizontalen oder vertikalen Bildachsen verlaufen. Auch bei diesen Messungen zeigte sich ein konsistentes Verhalten der berechneten Abstände.

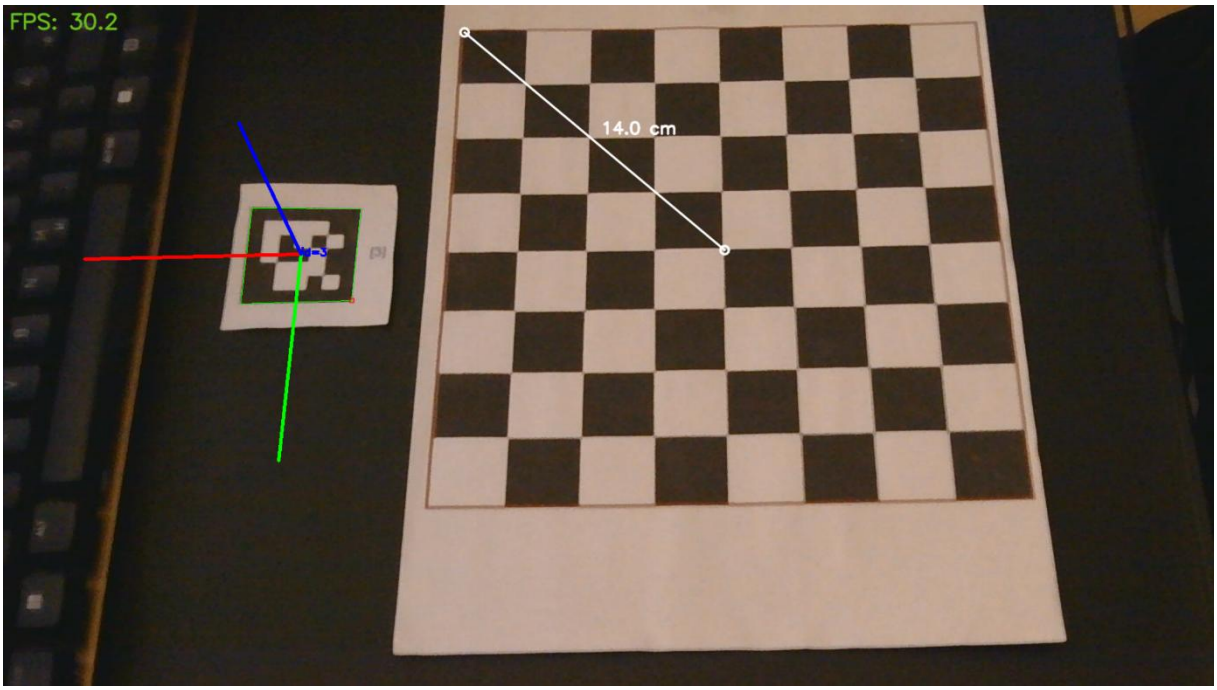


Abbildung 12: Diagonale Messung des Schachbrettmusters

Ähnlich wie bei den vorherigen Tests stellt die manuelle Auswahl der Messpunkte im Bild einen wesentlichen Einflussfaktor auf die Messgenauigkeit dar. Kleine Abweichungen von wenigen Pixeln können bereits zu geringfügigen Veränderungen der berechneten Distanz führen. Insgesamt bestätigen die Messungen am Schachbrettmuster die zuvor beobachtete Messgenauigkeit im Bereich weniger Prozent und zeigen, dass das System über unterschiedliche Messobjekte hinweg konsistente Ergebnisse liefert.

9.Fazit und Ausblick

9.1 Fazit

Ziel dieses Projekts war die Entwicklung eines kamerabasierten Messsystems, mit dem reale Abstände auf einer definierten Ebene mithilfe einer einzelnen Kamera bestimmt werden können. Hierfür wurde ein Python-basierter Softwareprototyp entwickelt, der Kamerakalibrierung, Marker-basierte Pose-Schätzung sowie geometrische Abstandsberechnung miteinander kombiniert.

Die durchgeführten Tests zeigen, dass das System stabil arbeitet und für den vorgesehenen Einsatzzweck geeignet ist. Der Kamerastream erreicht im praktischen Betrieb eine Bildrate von etwa 30 Bildern pro Sekunde bei einer Auflösung von 1920×1080 Pixeln. Dadurch ist eine flüssige Darstellung des Kamerabildes sowie eine direkte Benutzerinteraktion bei der Messung möglich. Auch die Marker-Erkennung erwies sich während der Tests als zuverlässig, solange der Marker in einem ausreichend frontalen Winkel zur Kamera positioniert ist.

Die eigentliche Messgenauigkeit wurde anhand von Referenzobjekten überprüft. Dazu wurden Messungen mit einem Meterstab sowie mit einem Schachbrettmuster durchgeführt. Die Ergebnisse zeigen eine gute Übereinstimmung zwischen den realen Referenzmaßen und den berechneten Abständen. In den meisten Fällen lagen die Abweichungen im Bereich von etwa 1 mm.

Auch bei unterschiedlichen Positionen im Bild sowie bei horizontalen, vertikalen und diagonalen Messungen zeigte das System ein konsistentes Verhalten. Dies bestätigt, dass die geometrische Rückprojektion der Bildpunkte auf die Marker-Ebene stabil funktioniert.

Die Auswertung der Messergebnisse zeigt außerdem, dass der größte Einflussfaktor auf die Messgenauigkeit nicht in der mathematischen Berechnung selbst liegt, sondern in der praktischen Bedienung des Systems. Insbesondere die manuelle Auswahl der Messpunkte im Bild kann zu kleinen Abweichungen führen, da es schwierig ist, exakt denselben Pixelpunkt zu treffen.

Insgesamt konnte im praktischen Test bei typischen Messstrecken eine Genauigkeit im Bereich weniger Prozent erreicht werden. Für ein System, das ausschließlich mit einer einzelnen RGB-Kamera arbeitet und ohne zusätzliche Tiefensensorik auskommt, stellt dies ein zufriedenstellendes Ergebnis dar.

Das entwickelte Programm erfüllt somit die in Kapitel 2 formulierten Projektziele. Es zeigt, dass mithilfe einer kalibrierten Kamera und einer bekannten Referenzstruktur reale Abstände aus einem Kamerabild berechnet werden können.

9.2 Ausblick

Obwohl das entwickelte System bereits funktionsfähig ist und stabile Messergebnisse liefert, bestehen verschiedene Möglichkeiten zur weiteren Verbesserung und Erweiterung. Ein möglicher Ansatz zur Erhöhung der Messgenauigkeit besteht in einer präziseren Kamerakalibrierung. Eine größere Anzahl von Kalibrierungsbildern sowie unterschiedliche Aufnahmewinkel könnten dazu beitragen, systematische Fehler in der Kameramatrix und den Verzerrungsparametern weiter zu reduzieren.

Darüber hinaus könnte die Benutzerinteraktion verbessert werden. Derzeit erfolgt die Auswahl der Messpunkte manuell per Maus. Eine automatische oder halbautomatische Punktbestimmung, beispielsweise durch Kantendetektion oder Feature-Erkennung, könnte die Genauigkeit der Messungen erhöhen und gleichzeitig die Bedienung vereinfachen.

Eine weitere Erweiterungsmöglichkeit besteht in der Verwendung mehrerer Referenzmarker. Durch die Kombination mehrerer Marker könnte die Lage der Messebene robuster bestimmt werden, insbesondere wenn einzelne Marker teilweise verdeckt sind oder sich am Rand des Bildes befinden.

Auch eine Erweiterung des Systems in Richtung dreidimensionaler Messungen wäre denkbar. Während das aktuelle System ausschließlich Abstände auf einer Ebene bestimmt, könnten durch zusätzliche Kameras oder Tiefensensoren vollständige 3D-Geometrien von Objekten erfasst werden. In industriellen Anwendungen werden hierfür häufig Mehrkamerasysteme oder laserbasierte Messverfahren eingesetzt. Durch mehrere Kameras kann ein Objekt aus unterschiedlichen Blickrichtungen erfasst werden, wodurch sich räumliche Strukturen präziser rekonstruieren lassen. Laserbasierte Verfahren, beispielsweise mit Laserlinien oder strukturiertem Licht, ermöglichen zusätzlich eine sehr genaue Erfassung von Oberflächengeometrien.

Solche Systeme werden beispielsweise in industriellen Montage- und Qualitätssicherungsanlagen eingesetzt, etwa bei Atlas Copco IAS, einem Partnerunternehmen zweier Autoren dieses Projektes. Dort kommen optische Messsysteme unter anderem zur Überwachung von Montageprozessen und zur automatischen Kontrolle von Bauteilgeometrien zum Einsatz. Im Vergleich zu dem in

dieser Arbeit entwickelten Prototypen ermöglichen solche Systeme eine deutlich höhere Messgenauigkeit sowie eine vollständige dreidimensionale Erfassung von Bauteilen.

Darüber hinaus könnte das System um automatische Messfunktionen erweitert werden. Beispielsweise wäre es möglich, bestimmte geometrische Merkmale von Bauteilen automatisch zu erkennen und deren Abstände zu bestimmen. Dadurch ließe sich das System perspektivisch auch für Anwendungen in der Qualitätssicherung einsetzen.

Zusammenfassend zeigt das Projekt, dass bereits mit vergleichsweise einfachen Mitteln ein funktionsfähiges kamerabasiertes Messsystem realisiert werden kann. Die entwickelte Lösung stellt eine solide Grundlage dar, auf der zukünftige Erweiterungen und weiterführende Anwendungen aufbauen können.

10.Literaturverzeichnis

apriltag (ohne Datum). Verfügbar unter: https://april.eecs.umich.edu/media/apriltag/?C=D;O=A (Zugegriffen: 6. März 2026).

Olson, E. (2011) „AprilTag: A robust and flexible visual fiducial system“, *2011 IEEE International Conference on Robotics and Automation*. *2011 IEEE International Conference on Robotics and Automation*, S. 3400–3407. Verfügbar unter: https://doi.org/10.1109/ICRA.2011.5979561.

Strobl, R. (2025) *Mechanik: Statik*. Wiesbaden: Springer Fachmedien. Verfügbar unter: https://doi.org/10.1007/978-3-658-47787-5.

Szeliski, R. (2022) *Computer vision: algorithms and applications*. Second edition. Cham: Springer (Texts in computer science).

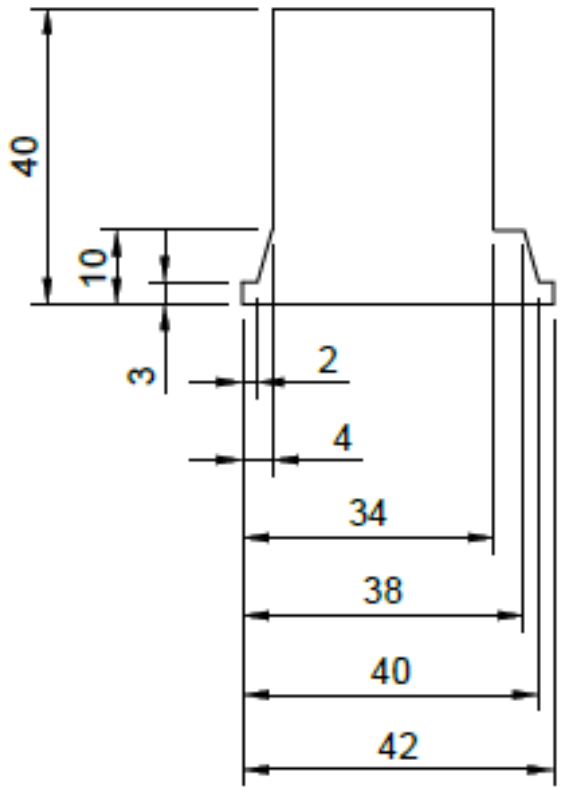
Zhang, Z. (2000) „A flexible new technique for camera calibration“, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), S. 1330–1334. Verfügbar unter: https://doi.org/10.1109/34.888718.

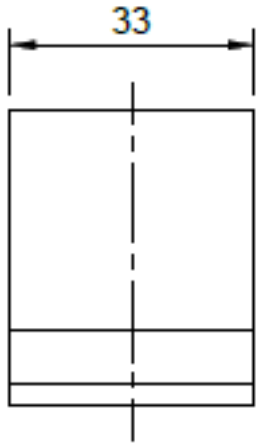
11.Bildquellen

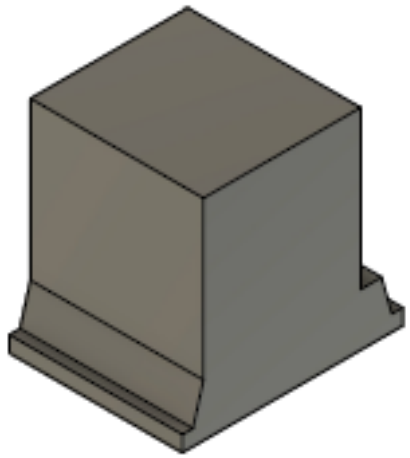
Deckblatt: Erstellt mit ChatGPT 5.4

12.Anhang

Zeichnungsableitung der Kamerahalterung







Dept.	Technical reference	Created by Tim Bosch 09.03.2026	Approved by
		Document type	Document status
		Title Kamerahalterung	DWG No.
			Rev. / Date of issue / Sheet 1/1